Optimization and Regularization

SGD, momentum, Adam, schedules, dropout, batch normalization

CSE752: Deep Neural Networks

Moin Moistakim
Department of CSE, School of Data and Sciences, BRAC University

Week 2 | Fall 2026

Today

Week 1 ended at one line: $\theta \leftarrow \theta - \eta \nabla_\theta L$. This week replaces that line, then keeps the fit from memorizing the training set.

Optimization

- Gradient descent and a step size that diverges
- Minibatch SGD
- Momentum and Nesterov
- Adam, bias correction, AdamW
- Step and cosine schedules

Regularization

- Train/test gap on a small set
- Weight decay
- Dropout, including `eval`
- Batch normalization and running statistics

Home Assignment W2 is an ablation of these choices on one MLP.

Learning objectives

By the end of this lecture you can:

1. Compute a gradient step on a quadratic and state when η diverges.
2. Write the PyTorch forms of momentum, Nesterov, and Adam, including the state each one stores.
3. Explain Adam's bias correction with the geometric-series identity.
4. Evaluate a step schedule and a cosine schedule at a given epoch.
5. Show that inverted dropout is unbiased, and say which statistics batch norm uses at test time.

Course outcomes: CO1 (training dynamics), CO3 (choose an optimizer and a regularizer).

One quadratic for the whole first half

Almost every optimizer slide uses the same scalar loss, so the arithmetic can be compared.

$$L(w) = \frac{1}{2}(w-4)^2, \qquad L'(w) = w - 4, \qquad L''(w) = 1.$$

The minimizer is $w^\star = 4$. Start at $w_0 = 0$ unless a slide says otherwise.

$$w_{t+1} - 4 = (1-\eta)\,(w_t - 4) \quad \text{for plain gradient descent.}$$

The error is multiplied by $1-\eta$ at every step. That factor is the whole story of step size on this loss.

Plain gradient descent

$$w_{t+1} = w_t - \eta\, L'(w_t).$$

With $L'(w) = w - 4$, $\eta = \frac{1}{2}$, and $w_0 = 0$:

$$w_1 = 0 - \tfrac{1}{2}(0 - 4) = 2,$$
$$w_2 = 2 - \tfrac{1}{2}(2 - 4) = 3,$$
$$w_3 = 3 - \tfrac{1}{2}(3 - 4) = \tfrac{7}{2}.$$

The errors are 4, 2, 1, $\frac{1}{2}$: each step cuts the distance to 4 in half, because $1 - \eta = \frac{1}{2}$.

The step that finishes in one iteration

$\eta = 1$ gives $w_1 = 0 - 1 \cdot (0 - 4) = 4$. On this quadratic the Hessian is 1, so $\eta = 1/L''(w)$ is an exact Newton step.

When the step size diverges

The same recurrence $e_{t+1} = (1 - \eta)e_t$ says:

- $|1 - \eta| < 1$, i.e. $0 < \eta < 2$, and the error goes to 0.
- $\eta = 2$ multiplies the error by -1: it oscillates and never settles.
- $\eta > 2$ multiplies by a number of absolute value greater than 1.

Try $\eta = 3$, $w_0 = 0$:

$$w_1 = 12, \qquad w_2 = -12, \qquad w_3 = 36.$$

The iterates leave any neighborhood of 4. In a network this is the run whose loss becomes `NaN`. The fix on this quadratic is a smaller η, not a new architecture.

A narrow valley and a flat one

Two coordinates, no cross term:

$$L(w_1, w_2) = \frac{1}{2} w_1^2 + \frac{100}{2} w_2^2, \qquad \nabla L = (w_1,\, 100\, w_2).$$

Eigenvalues of the Hessian: 1 and 100. Stability needs $\eta < 2/100 = 0.02$. Take the safe value $\eta = 0.01$ at $w = (1, 1)$:

$$w_1 \leftarrow 1 - 0.01 \cdot 1 = 0.99, \qquad w_2 \leftarrow 1 - 0.01 \cdot 100 = 0.$$

The sharp axis is finished in one step. The flat axis still has 99% of its error, and that factor 0.99 repeats. After 100 such steps the flat error is about 0.37. A single global η is stuck between "do not explode on the sharp axis" and "actually move on the flat one." Momentum and Adam are responses to that bind.

Minibatch SGD

The training loss is an average over N examples. Full-batch gradient descent uses all of them. A minibatch B replaces the gradient by

$$g_B = \frac{1}{|B|} \sum_{i \in B} \nabla L_i(w).$$

$$w \leftarrow w - \eta \, g_B.$$

- $\mathbb{E}[g_B] = \nabla L$ when the batch is drawn uniformly, so the update is unbiased for the full loss.
- Variance shrinks as $|B|$ grows. Batch size 1 is noisiest; batch size N is ordinary gradient descent.
- One epoch $\approx N/|B|$ updates. Home Assignment uses 128.

Worked example: two examples, one parameter

$$L(w) = \frac{1}{2} \cdot \frac{1}{2} \sum_{i=1}^{2} (w - y_i)^2, \quad y_1 = 1,\ y_2 = 3.$$

Then $L'(w) = w - 2$, and the minimizer is the mean $w^\star = 2$. Per-example derivatives at a point w are $w - y_i$.

Start at $w = 0$, use $\eta = 1$.

- Full batch: $g = \frac{1}{2}\big((0-1) + (0-3)\big) = -2$, so $w \leftarrow 2$. One step, done.
- Example 1 only: $g = -1$, so $w \leftarrow 1$.
- Then example 2 at $w = 1$: $g = 1 - 3 = -2$, so $w \leftarrow 3$.

The stochastic path visits $0 \to 1 \to 3$ and has already stepped past 2. The noise is not a bug in the arithmetic. It is the minibatch.

Polyak momentum, as PyTorch stores it

Keep a velocity v, with $v_0 = 0$ and momentum coefficient $\mu \in [0, 1)$. PyTorch's `SGD(momentum=`μ`)` does

$$v_{t+1} = \mu v_t + g_t,$$
$$w_{t+1} = w_t - \eta\, v_{t+1},$$

where g_t is the minibatch gradient. The extra state is one buffer the same shape as w.

Goodfellow writes $v \leftarrow \mu v - \eta g$ and $w \leftarrow w + v$. That is the same trajectory: his velocity equals $-\eta$ times the buffer above. When you read the book and the docs side by side, match where η sits before comparing formulas.

Momentum on the quadratic

Same loss, $g = w - 4$, $\eta = \frac{1}{2}$, $\mu = \frac{1}{2}$, $w_0 = 0$, $v_0 = 0$.

$$g_0 = -4, \qquad v_1 = -4, \qquad w_1 = 0 - \tfrac{1}{2}(-4) = 2,$$
$$g_1 = -2, \qquad v_2 = \tfrac{1}{2}(-4) + (-2) = -4, \qquad w_2 = 2 - \tfrac{1}{2}(-4) = 4.$$

Plain gradient descent with the same η was at 3 after two steps, then 3.5, 3.75, ... Momentum is at the minimizer on step 2, because the second velocity still carries the first gradient of -4.

What μ is doing

$\mu = 0$ is plain SGD. μ near 1 remembers old gradients for a long time, which is exactly the accumulation the flat axis of the valley needs. It can also overshoot once the gradient has changed sign. $\mu = 0.9$ is the usual default, not a theorem.

Nesterov on the same numbers

PyTorch's `nesterov=True` evaluates the gradient at the current weights (as usual in the training loop) and then applies

$$v \leftarrow \mu v + g, \qquad w \leftarrow w - \eta\,(g + \mu v).$$

Same start, $\eta = \mu = \frac{1}{2}$, $g_0 = -4$:

$$v_1 = -4, \qquad g + \mu v = -4 + \tfrac{1}{2}(-4) = -6, \qquad w_1 = 0 - \tfrac{1}{2}(-6) = 3.$$

Classical momentum reached 2 on this step; Nesterov reaches 3. One step later:

$$g_1 = -1, \quad v_2 = -3, \quad g + \mu v = -\tfrac{5}{2}, \quad w_2 = 3 + \tfrac{5}{4} = \tfrac{17}{4} = 4.25.$$

It overshoots 4 by a quarter. The lookahead is more aggressive, not magically exact.

Why a second moment

Momentum smooths the gradient, then multiplies by one global η. In the valley, the sharp coordinate wanted a small step and the flat coordinate wanted a large one. Adam keeps a running average of g and of $g \odot g$, and scales each coordinate by its own root-mean-square size.

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2)\, g_t \odot g_t, \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \\
w_t &= w_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.
\end{aligned}
$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. State: two buffers, m and v, each the shape of w.

Where the bias correction comes from

Unfold the first-moment recurrence. With $m_0 = 0$,

$$m_t = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i.$$

If every g_i equals the same vector g, the sum is geometric:

$$\sum_{j=0}^{t-1} \beta_1^j = \frac{1 - \beta_1^t}{1 - \beta_1}, \qquad m_t = g\,(1 - \beta_1^t).$$

Dividing by $1 - \beta_1^t$ recovers g. Early in training β_1^t is not small ($\beta_1 = 0.9$ still has $0.9^{10} \approx 0.35$), so the raw buffer m_t is much smaller than g. The same algebra applies to v_t with $g \odot g$ and β_2. That division is the bias correction. It is largest at step 1 and dies once t is large.

One Adam step, in fractions

Scalar, constant $g = 2$, $\beta_1 = \beta_2 = \frac{1}{2}$, $\eta = \frac{1}{5}$, $\varepsilon = 0$, $m_0 = v_0 = 0$. Step $t = 1$:

$$m_1 = \tfrac{1}{2} \cdot 2 = 1, \qquad v_1 = \tfrac{1}{2} \cdot 4 = 2,$$

$$\hat{m}_1 = \frac{1}{1 - \frac{1}{2}} = 2, \qquad \hat{v}_1 = \frac{2}{1 - \frac{1}{2}} = 4.$$

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = \frac{2}{2} = 1, \qquad \Delta w = \eta \cdot 1 = \frac{1}{5}.$$

The corrected step has size η, the full learning rate, because $|g|/|g| = 1$.
Without the correction one would have used $m_1/\sqrt{v_1} = 1/\sqrt{2}$, and the step would have been $\eta/\sqrt{2} \approx 0.141$ instead of 0.200. The missing factor is exactly $1 - \beta^t$ at $t = 1$, which is $\frac{1}{2}$ in m and, after the square root, $\sqrt{1/2}$ in the ratio.

AdamW is not Adam plus a smaller η

PyTorch's `Adam(weight_decay=`λ`)` adds λw to the gradient *before* the moments. On a scalar at step 1, bias correction then cancels that addition in the ratio $\hat{m}/\sqrt{\hat{v}}$, and the step collapses to η times a sign. `AdamW` applies decay outside the adaptive ratio:

$$w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon} - \eta\lambda w.$$

Numbers: $w = 2$, $g = 1$, $\lambda = 0.1$, $\eta = 0.1$, step 1, $\varepsilon = 0$.

- Coupled Adam: $\tilde{g} = 1.2$, ratio 1, step 0.1, new $w = 1.9$. Decay disappeared.
- AdamW: adaptive step 0.1, decay $0.1 \cdot 0.1 \cdot 2 = 0.02$, new $w = 1.88$.

Home Assignment's stretch uses AdamW at 10^{-4}. It is not interchangeable with dropout: one shrinks weights, the other randomly drops activations.

Step decay, matching the assignment

A constant η that was safe at random initialization is often too large once the loss is smaller. `StepLR(step_size=5, gamma=0.1)` multiplies η by 0.1 every 5 scheduler steps.

$$\eta_0 = 0.1 \xrightarrow{\text{5 steps}} 0.01 \xrightarrow{\text{5 more}} 0.001.$$

Call `sched.step()` *after* `opt.step()`, once per epoch in the assignment. During the first five epochs the learning rate is still 0.1.

Cosine annealing

Over a horizon of T scheduler steps, with floor $\eta_{\min}$,

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2}\left(1 + \cos\frac{\pi t}{T}\right), \qquad t = 0, 1, \ldots, T.$$

For the assignment, $\eta_{\max} = 0.1$, $\eta_{\min} = 0$, $T = 8$:

t	0	4	8
$\cos(\pi t/T)$	1	0	-1
η_t	0.10	0.05	0

The decay is smooth. Nothing in the formula knows about the loss. It only knows the epoch.

The gap the assignment makes you plot

On 1,000 Fashion-MNIST images for 25 epochs, a $784 \to 128 \to 10$ net can drive training accuracy far above test accuracy. That gap is overfitting: the minimizer of the training loss is not a minimizer of the loss you care about.

- More data, or a smaller net, shrinks the gap from the capacity side.
- Early stopping is a schedule on the *number of steps*, chosen by the validation curve, not by the training loss.
- Weight decay, dropout, and batch norm change the objective or the noise, without deleting the training set.

Plot train and test on the same axes. A falling training loss alone cannot show overfitting.

Weight decay

Add a quadratic penalty, $\lambda > 0$:

$$\widetilde{L}(w) = L(w) + \frac{\lambda}{2}\|w\|_2^2, \qquad \nabla\widetilde{L} = \nabla L + \lambda w.$$

One gradient step is

$$w \leftarrow w - \eta(\nabla L + \lambda w) = (1 - \eta\lambda)\, w - \eta\nabla L.$$

Every coordinate is multiplied by $1 - \eta\lambda$ and then nudged by the data gradient. Weights are pulled toward 0 whether or not the data gradient is large. Biases are often left out of the penalty; PyTorch's `weight_decay` does not make that distinction unless you use two parameter groups.

Dropout

PyTorch's `Dropout(p)` uses inverted dropout. During training, for keep-probability $q = 1 - p$,

$$m_i \sim \text{Bernoulli}(q), \qquad h'_i = \frac{m_i}{q}\, h_i, \qquad \mathbb{E}[h'_i] = h_i.$$

The $1/q$ is what makes the expectation match the un-dropped activation, so test time can use the identity.

Example. $p = 0.5$, so $q = 0.5$ and the scale is 2. Activations (2, 0, 4, 6), mask (1, 0, 1, 0):

$$h' = (4,\ 0,\ 8,\ 0).$$

The assignment checkpoint

Dropout is on in `model.train()` and off in `model.eval()`. Leaving it on at test time makes every prediction a fresh random subnetwork. Accuracy bounces, and with inverted dropout you are also not using the $1/q$ compensation the training loss saw.

Batch normalization

For one unit, on a minibatch $x_1, \ldots, x_m$, training computes

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2,$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta.$$

γ and β are learned. They can undo the normalization if that is what the loss wants. The useful part is that the *input* to the nonlinearity is recentered and rescaled every batch, so a large learning rate is less likely to push the next layer into a saturated or exploding regime.

Batch norm, four numbers

Batch $x = (1, 3, 5, 7)$, $\varepsilon = 0$, $\gamma = 1$, $\beta = 0$.

$$\mu_B = 4, \qquad \sigma_B^2 = \frac{(-3)^2 + (-1)^2 + 1^2 + 3^2}{4} = 5.$$

$$\hat{x} = \frac{(-3,\ -1,\ 1,\ 3)}{\sqrt{5}}.$$

The four outputs have mean 0 and mean-square 1.

Home Assignment inserts it as `Linear(784, 128)`, `BatchNorm1d(128)`, `ReLU`. Normalize the pre-activation, then apply ReLU. The assignment also asks you to try SGD at $\eta = 0.1$ and at 0.3: the second value is the one that often diverges *without* this layer and survives *with* it.

Test time uses the running statistics

A test point has no training batch. Batch norm keeps exponential averages, with PyTorch's default momentum $\alpha = 0.1$:

$$\mu_{\text{run}} \leftarrow (1-\alpha)\,\mu_{\text{run}} + \alpha\,\mu_B,$$
$$\sigma^2_{\text{run}} \leftarrow (1-\alpha)\,\sigma^2_{\text{run}} + \alpha\,\sigma^2_{\text{unbiased}}.$$

At evaluation, $\hat{x} = (x - \mu_{\text{run}})/\sqrt{\sigma^2_{\text{run}} + \varepsilon}$, then the same γ, β. The map is deterministic.
If the running mean starts at 0 and the first batch has $\mu_B = 4$,

$$\mu_{\text{run}} = 0.1 \cdot 4 = 0.4.$$

That is not 4. Early checkpoints still have cold running statistics. The assignment checkpoint is the principle: do not recompute μ_B from the test loader.

What to remember in one table

Method	Extra state	One-line update
SGD	none	$w \leftarrow w - \eta g$
Momentum	velocity v	$v \leftarrow \mu v + g,\ w \leftarrow w - \eta v$
Nesterov	velocity v	$w \leftarrow w - \eta(g + \mu v)$ after the same v
Adam	m and v	$w \leftarrow w - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$
AdamW	m, v, plus λ	Adam step, then $-\eta\lambda w$ outside it
Step / cosine	epoch counter	replace η before the update
Dropout	mask, train-only	$h \leftarrow m \odot h/(1-p)$
Batch norm	$\gamma, \beta, \mu_{\text{run}}, \sigma_{\text{run}}$	normalize, then affine

The loop the assignment is varying

```
for epoch in range(epochs):
    model.train()
    for x, y in loader_tr:
        opt.zero_grad()
        loss = criterion(model(x), y)   # CrossEntropyLoss
        loss.backward()
        opt.step()
    sched.step()                         # only in the schedule runs
    model.eval()                         # dropout off, batch norm uses buffers
    # compute test accuracy; do not call backward here
```

The four optimizer runs differ only in `opt`. The dropout run differs in the module and in remembering `eval`. If those two are mixed, the ablation table does not measure the thing you think it measures.

In-class problems

Same quadratic as the lecture where a problem says so: $L(w) = \frac{1}{2}(w - 4)^2$. Exact fractions.

1. Gradient descent, two step sizes. 6 minutes
2. Momentum against plain GD. 8 minutes
3. One Adam step, with and without bias correction. 8 minutes
4. Batch norm on four numbers, plus the test-time question. 8 minutes

Worked solutions are linked from each slide. The notes file has the geometric-series proof, the Nesterov second step, and two further problems (SGD path, dropout expectation).

Problem 1. Step size

$L(w) = \frac{1}{2}(w-4)^2$, $w_0 = 0$.

1. Take $\eta = 1$. Compute w_1.
2. Take $\eta = 3$. Compute w_1 and w_2.
3. For which $\eta > 0$ does the error $w_t - 4$ tend to 0?

▸ Solution 1

Problem 2. Momentum

Same loss, $w_0 = 0$, $v_0 = 0$, $\eta = \frac{1}{10}$, $\mu = \frac{9}{10}$. Use $v \leftarrow \mu v + g$ and $w \leftarrow w - \eta v$, with $g = w - 4$.

1. Compute w_1 and w_2.
2. Compute the plain gradient-descent iterate with the same η after two steps.
3. Which one is closer to 4?

▸ Solution 2

Problem 3. Adam

Scalar Adam, $g = 2$ constant, $\beta_1 = \beta_2 = \frac{1}{2}$, $\eta = \frac{1}{5}$, $\varepsilon = 0$, $m_0 = v_0 = 0$, $t = 1$.

1. Compute m_1, v_1, $\hat{m}_1$, $\hat{v}_1$.
2. Compute the parameter step $\eta\, \hat{m}_1 / \sqrt{\hat{v}_1}$.
3. Compute the step you would get from $m_1 / \sqrt{v_1}$ with no bias correction.

▸ Solution 3

Problem 4. Batch norm

One unit, batch $x = (2, 4, 4, 6)$, $\varepsilon = 0$, $\gamma = 1$, $\beta = 0$.

1. Compute μ_B, the biased variance σ_B^2, and $\hat{x}$.
2. The running mean starts at 0, and the buffer momentum is 0.1. What is μ_{run} after this batch?
3. At test time, which mean and variance are used, and why is μ_B of this batch the wrong one for a single test image?

▸ Solution 4

What this week asks of you

- **Read.** Goodfellow Ch. 7–8. Prince Ch. 6 and Ch. 11. Adam: Kingma and Ba (2015). Dropout: Srivastava et al. (2014). Batch norm: Ioffe and Szegedy (2015).
- **Home Assignment W2.** One MLP, four optimizers, step versus cosine, then a 1,000-image overfit run with dropout and with batch norm. Fill the ablation table from your seed, not from a blog post.
- **Checkpoints on that sheet** are today's formulas: write momentum in two lines, name Adam's two buffers, say why evaluation must not recompute batch statistics, and say what dropout-at-test does.

Equations worth leaving with

$$w \leftarrow w - \eta g$$

$$v \leftarrow \mu v + g, \qquad w \leftarrow w - \eta v$$

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g \odot g$$

$$\hat{m} = \frac{m}{1-\beta_1^t}, \qquad w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}$$

$$\mathbb{E}[h \odot m/(1-p)] = h$$

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \quad \text{test } \mu = \mu_{\text{run}}$$

References

- Goodfellow, Bengio, Courville. *Deep Learning*, Ch. 7–8.
- Prince. *Understanding Deep Learning*, Ch. 6 and Ch. 11. `https://udlbook.github.io/udlbook/`
- Kingma and Ba. "Adam: A Method for Stochastic Optimization." ICLR, 2015.
- Loshchilov and Hutter. "Decoupled Weight Decay Regularization." ICLR, 2019 (AdamW).
- Srivastava et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *JMLR*, 2014.
- Ioffe and Szegedy. "Batch Normalization." ICML, 2015.

Solution 1

$g(w) = w - 4$.

1. $\eta = 1$: $w_1 = 0 - 1 \cdot (0 - 4) = 4$. That is the minimizer.
2. $\eta = 3$: $w_1 = 0 - 3 \cdot (-4) = 12$. Then $g(12) = 8$, so $w_2 = 12 - 3 \cdot 8 = -12$.
3. The error multiplies by $1 - \eta$ each step, so it tends to 0 iff $|1 - \eta| < 1$. For $\eta > 0$ that is $0 < \eta < 2$.

▸ Back to Problem 1

Solution 2

1. $g_0 = -4$, $v_1 = -4$, $w_1 = 0 - \frac{1}{10}(-4) = \frac{2}{5}$.

$$g_1 = \frac{2}{5} - 4 = -\frac{18}{5}, \qquad v_2 = \frac{9}{10}(-4) + \left(-\frac{18}{5}\right) = -\frac{36}{5}.$$

$$w_2 = \frac{2}{5} - \frac{1}{10}\left(-\frac{36}{5}\right) = \frac{2}{5} + \frac{18}{25} = \frac{28}{25} = 1.12.$$

2. Plain GD: $w_1 = \frac{2}{5}$, $w_2 = \frac{2}{5} - \frac{1}{10}\left(\frac{2}{5} - 4\right) = \frac{2}{5} + \frac{9}{25} = \frac{19}{25} = 0.76$.
3. Momentum is at 28/25, GD at 19/25. Distance to 4 is 72/25 versus 81/25. Momentum is closer.

Back to Problem 2

Solution 3

1. $m_1 = \frac{1}{2} \cdot 2 = 1$, $v_1 = \frac{1}{2} \cdot 4 = 2$, $\hat{m}_1 = 1/\frac{1}{2} = 2$, $\hat{v}_1 = 2/\frac{1}{2} = 4$.
2. $\hat{m}_1/\sqrt{\hat{v}_1} = 2/2 = 1$, so the step is $\frac{1}{5} \cdot 1 = \frac{1}{5}$.
3. Uncorrected: $1/\sqrt{2}$. The step is $\frac{1}{5} \cdot 1/\sqrt{2} = \frac{1}{5\sqrt{2}} \approx 0.141$, smaller than $\frac{1}{5} = 0.2$.

Back to Problem 3

Solution 4

1. $\mu_B = (2 + 4 + 4 + 6)/4 = 4$. Squared deviations: $4, 0, 0, 4$. Biased variance $\sigma_B^2 = 8/4 = 2$.

$$\hat{x} = \frac{(-2,\ 0,\ 0,\ 2)}{\sqrt{2}} = (-\sqrt{2},\ 0,\ 0,\ \sqrt{2}).$$

2. $\mu_{\text{run}} = 0.9 \cdot 0 + 0.1 \cdot 4 = 0.4$.
3. Test time uses μ_{run} and σ^2_{run}. A single test image does not define a training batch, and reusing this batch's $\mu_B = 4$ would make the prediction depend on training examples that are not the input.

Back to Problem 4